

Politechnika Świętokrzyska Wydział Elektrotechniki, Automatyki i Informatyki	
Wprowadzenie do komunikacji człowiek-komputer laboratorium (studia stacjonarne)	
Instrukcja laboratoryjna nr 5: Dokumentacja interfejsu użytkownika.	Opracowanie: Błażej Knap Data: 21.05.2026 r.

Niniejszy dokument stanowi kompletną dokumentację techniczną oraz instrukcję wdrożenia (instalacji) projektu Everbloom – interfejsu użytkownika w stylu farming sim / cozy RPG, zrealizowanego w środowisku Unity 6 LTS.

Dokument opisuje wymagania systemowe, strukturę plików kompilacji, zasoby zewnętrzne (assety) oraz krok po kroku wyjaśnia proces uruchamiania i wdrażania gry – zarówno z perspektywy gracza/testera (wersja skompilowana), jak i dewelopera (wersja źródłowa).

1. Docelowa Platforma i Charakterystyka Projektu

- Docelowa platforma po stronie klienta: Komputery osobiste PC (system operacyjny Microsoft Windows).
- Architektura: x86_64 (64-bit).
- Technologia wykonania: Silnik Unity 6 LTS z wykorzystaniem potoku renderowania URP (Universal Render Pipeline), systemu UI Toolkit oraz TextMeshPro.
- Metody sterowania: Klawiatura + Mysz.
 - Wybór slotów na pasku zadań: Klawisze 1 – 0
 - Ruch postaci: Standardowe klawisze ruchu (W, A, S, D)
 - Otwarcie ekwipunku: Klawisz E
 - Interakcja z obiektami: kliknięcie lewym przyciskiem myszy
 - Zamykanie dialogów / menu: wybrane opcji dialogowej.

2. Instrukcja Wdrożenia (Uruchomienie Gry)

A. Wersja Skompilowana (Dla Gracza / Testera)

Kompilacja gry znajduje się w folderze `Execute/`. Gra została zbudowana przy użyciu środowiska uruchomieniowego Mono w Unity.

Plik uruchomieniowy:

Głównym plikiem wykonywalnym gry jest:
`.\Everbloom-Game-Interface1\Execute\Everbloom.exe`

Kroki instalacji/uruchomienia:

1. Pobierz lub sklonuj całe repozytorium na dysk lokalny.
2. Przejdź do folderu `Execute/`.
3. Uruchom plik `Everbloom.exe`.

UWAGA! Wymagana spójność katalogu: Nie wolno przenosić samego pliku `Everbloom.exe` poza jego folder macierzysty. Do poprawnego działania gry niezbędne są powiązane pliki i foldery znajdujące się w tym samym katalogu:

- `Everbloom_Data/` – zawiera zasoby gry, skompilowane sceny i pliki konfiguracyjne.
- `MonoBleedingEdge/` – zawiera środowisko uruchomieniowe C# Mono.
- `UnityPlayer.dll` – biblioteka silnika Unity.
- `DirectML.dll` – biblioteka Direct Machine Learning (wykorzystywana przez silnik Unity 6 do obsługi silnika wnioskowania AI/Inference).
- `UnityCrashHandler64.exe` – moduł raportowania błędów.
- `D3D12/` – biblioteki sterownika graficznego DirectX 12.

B. Wersja Deweloperska (Dla Programisty / Otwarcie w Edytorze)

Wdrożenie projektu w środowisku deweloperskim wymaga instalacji silnika Unity oraz importu odpowiednich pakietów.

Wymagania przygotowawcze:

1. Zainstaluj Unity Hub.
2. Zainstaluj wersję silnika Unity 6 LTS (zalecana i potwierdzona wersja: `6000.3.14f1`).
3. Zainstaluj edytor kodu wspierający Unity (np. *Visual Studio 2022* z pakietem roboczym Unity lub *JetBrains Rider*).

Procedura wdrożenia projektu:

1. Otwórz Unity Hub.
2. Kliknij przycisk Add -> Add project from disk.
3. Wskaż podfolder `Everbloom/` znajdujący się w głównym katalogu repozytorium (zawiera on pliki `Assets/`, `Packages/` oraz `ProjectSettings/`).
4. Zatwierdź otwarcie projektu. Unity automatycznie pobierze wymagane zależności określone w pliku manifestu i wygeneruje foldery tymczasowe (`Library/`, `Temp/`).
5. Po załadowaniu projektu w edytorze upewnij się, że sceny są dodane w oknie Build Settings (`File -> Build Settings`) w następującej kolejności:

- i. `MainMenu` (Index 0) – *wymagane, ponieważ inicjalizuje singletony i globalne systemy*
 - ii. `NewGameMenu` (Index 1)
 - iii. `SettingsMenu` (Index 2)
 - iv. `GameScene` (Index 3)
 - v. `Calendar` (Index 4)
 - vi. `DaySummary` (Index 5)
6. Aby przetestować grę w edytorze, otwórz scenę `MainMenu` i wciśnij przycisk Play.

3. Wymagania Techniczne i Systemowe

Projekt został wdrożony i przetestowany na nowoczesnej architekturze sprzętowej. Poniżej przedstawiono wymagania systemowe oraz konfigurację referencyjną.

A. Potwierdzona Konfiguracja Sprzętowa (Środowisko Testowe)

Gra działa w pełni płynnie i bezbłędnie na poniższej konfiguracji sprzętowej użytkownika:

Komponent	Specyfikacja Środowiska Testowego
System Operacyjny	Windows 11 Home (Wersja: 10.0.26200, Kompilacja: 26200)
Procesor (CPU)	AMD Ryzen 5 5500 (6 rdzeni fizycznych, 12 wątków logicznych, taktowanie bazowe 3.6 GHz)
Karta Graficzna (GPU)	NVIDIA GeForce RTX 4060 (z pełną obsługą DirectX 12 i sprzętowego Ray Tracingu)
Pamięć RAM	64 GB
Wersja Unity	Unity 6 LTS (Dokładna wersja edytora: 6000.3.14f1)

B. Sugerowane Wymagania Systemowe (PC)

Dzięki zastosowaniu zoptymalizowanego potoku renderowania dwuwymiarowego (URP 2D), wymagania sprzętowe gry są relatywnie niskie, co pozwala na jej uruchomienie na szerokiej gamie urządzeń.

Wymagania Minimalne:

- System operacyjny: Windows 10 (64-bit)
- Procesor (CPU): Intel Core i3 / AMD Ryzen 3 (lub odpowiednik dwurdzeniowy z taktowaniem min. 2.5 GHz)
- Pamięć RAM: 4 GB RAM
- Karta graficzna (GPU): Zintegrowana karta Intel HD Graphics 620 / AMD Radeon Vega 3 (z obsługą DirectX 11 / Shader Model 5.0)
- Miejsce na dysku: Około 150 MB wolnego miejsca
- Karta dźwiękowa: Zgodna z DirectX

Wymagania Rekomendowane:

- System operacyjny: Windows 10 / 11 (64-bit)
- Procesor (CPU): Intel Core i5 / AMD Ryzen 5 lub nowszy
- Pamięć RAM: 8 GB RAM lub więcej
- Karta graficzna (GPU): Dedykowana karta NVIDIA GeForce GTX 1050 / AMD Radeon RX 560 lub lepsza (z obsługą DirectX 12)
- Miejsce na dysku: Około 150 MB wolnego miejsca (rekomendowany dysk SSD dla szybszego ładowania scen)

4. Zależności i Zastosowane Assety Zewnętrzne

Gra Everbloom korzysta z darmowych zasobów graficznych i programistycznych, które zostały wbudowane w strukturę projektu. Poniżej znajduje się zestawienie pakietów instalacyjnych oraz darmowych assetów wraz z licencjami i autorami:

Pakiety Unity (Package Manager Dependencies):

1. com.unity.inputsystem (v1.19.0) – Nowy, modularny system obsługi sterowania w Unity.
2. com.unity.render-pipelines.universal (v17.3.0) – Universal Render Pipeline (URP) zapewniający nowoczesne oświetlenie 2D, optymalizację graficzną oraz postprocessing.
3. com.unity.ugui (v2.0.0) – Zintegrowany system interfejsu (w tym wsparcie dla nowszego UI Toolkit).

4. `com.unity.ai.inference` (v2.6.1) & `com.unity.ai.assistant` – Pakiety Unity AI wykorzystujące silnik wnioskowania (Inference Engine) oraz biblioteki uczenia maszynowego (związane z DirectML).

Darmowe Assety i Licencje (Assets Folder):

- Fantasy Wooden GUI Free (Unity Asset Store)
 - *Zastosowanie:* Elementy graficzne interfejsu użytkownika (drewniane tła paneli, ozdobne ramki, tekstury przycisków, okna dialogowe).
- Inventory System – Kinnly (Unity Asset Store)
 - *Zastosowanie:* Gotowe skrypty logiki ekwipunku, mechanika przeciągania przedmiotów (Drag & Drop), dzielenie przedmiotów prawym przyciskiem myszy, system paska szybkiego wyboru (Toolbar/Hotbar), a także interakcje środowiskowe (smutowanie rud w piecu, ścinanie drzew, kopanie rud).
- Pixel Cursors (Unity Asset Store)
 - *Zastosowanie:* Zestaw stylizowanych, pikselowych kursorów myszy zmieniających się w zależności od najechania na elementy interaktywne.
- Pixel Skies DEMO (Unity Asset Store)
 - *Zastosowanie:* Pikselowe tła nieba używane do budowania nastroju w grze oraz wizualizacji kalendarza i menu głównego.
- TopDown 2D RPG BE3 (Unity Asset Store)
 - *Zastosowanie:* Grafiki kafelków mapy (tilesety), animacje postaci gracza, grafiki otoczenia (drzewa, kamienie, surowce), stanowiące oprawę wizualną sceny rozgrywki (`GameScene`).
- TinyHealthSystem autorstwa ZombiSoft (Assets/ZombiSoft)
 - *Zastosowanie:* Kompletny i lekki komponent paska zdrowia i zarządzania punktami życia postaci.
- Pixel Art Farm Tools autorstwa Anonymous Side Projects (Assets/Anonymous Side Projects)
 - *Zastosowanie:* Grafiki pikselowych narzędzi rolniczych (motyki, konewki, siekiery itp.) wykorzystywane w pasku narzędziowym.
- RPG Pixel Art Icons (Gold Series & 420 Icons) autorstwa 7soul1 (DeviantArt)
 - *Zastosowanie:* Ikony przedmiotów w ekwipunku oraz na pasku szybkiego wyboru.
 - *Licencja:* CC0 (Public Domain) – wolne do użytku komercyjnego i niekomercyjnego. [Link do źródła](#).

5. Architektura Systemu UI i Rozwiązywanie Problemów

Innowacje i Skrypty Pomocnicze UI:

- `GlobalInputManager.cs` – Klasa typu Singleton ze statyczną instancją, która nie ulega zniszczeniu przy zmianie sceny (`DontDestroyOnLoad`). Obsługuje globalne skróty klawiszowe (np. wyjście z gry za pomocą `Escape`).
- `GlobalUIFixer.cs` – Skrypt automatyzujący konfigurację interfejsu. Po załadowaniu sceny samodzielnie weryfikuje poprawność modułu `EventSystem` (tworzy go, jeśli go brakuje), wyłącza blokowanie kliknięć (`Raycast Target`) na elementach czysto tekstowych `TextMeshPro` (co poprawia wydajność i zapobiega błędom klikania) oraz automatycznie przypisuje skrypt efektów hover do wszystkich przycisków.
- `ButtonHoverEffect.cs` – Zapewnia spójne mikro-animacje przycisków: płynne przeskalowanie przy najechaniu kursorem, zmianę koloru tintu oraz animację wizualnego wciśnięcia.
- `SettingsManager.cs` – Odpowiada za konfigurację audio. Zapisuje ustawienia głośności i stan wyciszenia za pomocą klasy `PlayerPrefs`, co gwarantuje, że preferencje użytkownika są trwale zachowywane pomiędzy kolejnymi uruchomieniami gry (sesjami).

Rozwiązywanie Typowych Problemów (FAQ):

1. Problem: Przycisk "Wyjdź" w menu głównym nie działa.
 - **Wyjaśnienie:** Funkcja `Application.Quit()` w skrypcie `MainMenuManager.cs` zamyka skompilowaną aplikację `.exe`. Wewnątrz edytora Unity funkcja ta celowo nie powoduje wyjścia (działa jedynie instrukcja wyłączenia trybu Play w kodzie deweloperskim). Jest to poprawne zachowanie silnika Unity.
2. Problem: Ekrany kalendarza lub podsumowania dnia nie chcą się załadować.
 - **Rozwiązanie:** Sprawdź, czy nazwy scen w skrypcie `SceneLoader.cs` lub `SceneController.cs` pokrywają się dokładnie z nazwami scen w projekcie oraz czy sceny te zostały dodane do okna Build Settings.
3. Problem: Myszka nie reaguje na przyciski interfejsu.
 - **Rozwiązanie:** Skrypt `GlobalUIFixer` dba o obecność obiektu `EventSystem`. Upewnij się, że w scenie nie ma zduplikowanych modułów wejścia oraz że kamera główna posiada komponent `Physics2DRaycaster`, jeśli interakcja dotyczy fizycznych obiektów `Sprite` na mapie (np. wyzwalanych przez `SpriteHoverEffect`).